JOB CORE · REACT QUERY · PHASES 1–2

React Query — Lesson 01: Fundamentals

Server-State Architecture, Caching Lifecycle, useQuery Setup, and Query Keys

Source Lesson: Fundamentals · Classroom & Self-Study Deck

HOW TO USE THIS · LESSON ROADMAP

Master Core Topics First, Toggle Mention Later

Study Vault Review Workflow

- **Checklist PDF Review:** Scan every slide like a topic checklist before coding.
- **Core vs Mention Tiers:** Know Core topics now; explore Mention items later.
- **Toggle Study Vault:** Open Fundamentals in Study Vault when a topic feels unclear.
- **Full lesson.md Labs:** Complete practice exercises and Project 1 in lesson.md.



1. INTRODUCTION · CORE (KNOW THESE)

React Query Replaces Fragile useEffect Fetching With Server Cache

What Is TanStack Query & Why Use It

- **Async State Manager:** Fetches, caches, and syncs remote server state.
- **Problems Solved:** Deduplicates requests and auto-updates stale cache.

Manual useEffect Fetching Pain

- **Boilerplate Flags:** Hand-wiring loading, error, and data on every view.
- **Race Conditions:** Stale responses overwrite UI; remounts trigger refetches.

Dimension	Client State (UI Memory)	Server State (Remote API & Cache)
Location & Control	Lives in browser memory; 100% owned by local UI.	Stored remotely; shared and mutated by other users.
Freshness & Sync	Synchronous and always current via useState.	Asynchronous; turns stale and needs revalidation.
Ideal Tooling	useState, useReducer, Context, Zustand, or Redux.	React Query (TanStack Query) for cache and sync.

1. INTRODUCTION · MENTION (LEARN LATER)

Pair React Query With Fetchers and UI Stores

How React Query Fits the Ecosystem

- **vs Redux:** Redux holds client UI state; React Query replaces async thunks.
- **vs RTK Query:** RTK Query couples to Redux; React Query works standalone.
- **vs Axios / Fetch:** Axios executes HTTP calls; React Query caches the response.
- **When to Use / Avoid:** Use for remote APIs and sync; skip on static-only sites.



Stale-While-Revalidate Serves Cache Instantly and Syncs Silently

Lifecycle (Core)

- **Fetch · Cache · Sync:** Fetches remote data, stores it by key, and updates subscribers.
- **Shared Ownership:** Multiple components read one cache entry without extra network calls.

Freshness (Core)

- **Fresh Cache Hit:** Fresh data returns immediately without hitting the network.
- **Stale & Refetch:** Stale data renders fast while refetching in the background.

Sync Flow (Note)

- **Cache Invalidation:** Marks cached keys stale after edits so UI auto-refreshes.
- **Architecture Flow:** Component mounts -> observer checks cache -> background sync.

4–5. SETUP & USEQUERY · CORE (KNOW THESE)

Wrap Root in QueryClient and Call useQuery

Four Building Blocks of Every Query

- **Client & Provider:** Create new QueryClient() once and wrap <App /> at root.
- **queryKey + queryFn:** Pass a unique key array ['users'] and a Promise callback.
- **Initial vs Background:** First fetch loads fresh; remounts show cache while syncing.
- **Manual refetch():** Call refetch() on user actions to force an immediate update.



4–5. LOADING FLAGS (CORE) & CONFIG (MENTION)

Separate First-Load Spinners From Background Sync States

Status Flag (Core)	When It Is True	UI Pattern for Students
isPending	Cache has no data yet (status === 'pending').	Render initial skeleton or empty placeholder.
isFetching	Any network request is active (initial or background).	Show subtle top bar or 'Refreshing...' badge.
isLoading	isPending && isFetching (first active fetch).	Show primary full-section loading spinner.

Mention: Global Defaults & Return Fields

- **Global Defaults:** Configure staleTime, gcTime, and retry on QueryClient.
- **Return Fields:** Read data, error, isError, isSuccess, and status.

Mention: React Query DevTools Setup

- **Cache Inspector:** Mount <ReactQueryDevtools /> inside QueryClientProvider.
- **Live Debugging:** Inspect fresh, stale, and inactive queries in development.

6–7. KEYS & QUERYFN · CORE + MENTION

Use Unique Query Keys and Promise queryFns

Cache Addressing & Promise Rules

- **Key Identity (Core):** Keys index the cache; nest params like ['users', id].
- **Promise & Errors (Core):** queryFn returns a Promise and throws on failed requests.
- **Cancellation (Core):** Pass AbortSignal from query context to cancel stale fetches.
- **Mention Deep Dives:** Key factories, Fetch vs Axios (!res.ok), and context fields.



PRACTICE & REVIEW · FULL LESSON.MD

Apply Phases 1–2 in Full lesson.md Practice and Project 1

Practice & Labs

- **useEffect vs React Query:** Refactor a fragile manual useEffect fetch into useQuery.
- **Project 1: User Directory:** Build a searchable user list with parameterized keys and refetch.

Self-Check Quiz

- **Core Concept Check:** Contrast server vs client state and fresh vs stale cache.
- **Code & Key Audit:** Verify isPending vs isFetching and proper queryKey params.

Phase 3 Preview

- **Next in Phase 3:** Advance to custom staleTime, gcTime, and mutations.
- **If You Missed Class:** Read full lesson.md in Study Vault and toggle Mention blocks.

Practice Roadmap · Located at the bottom of full lesson.md in Study Vault

LESSON 01 COMPLETE · STUDY VAULT

Keys Identify the Cache. React Query Synchronizes the Server.

Open lesson.md in Study Vault for Project 1: User Directory.